

Bibliothicc – Datenbankdokumentation

ERM, relationales Modell und Normalisierungsnachweis

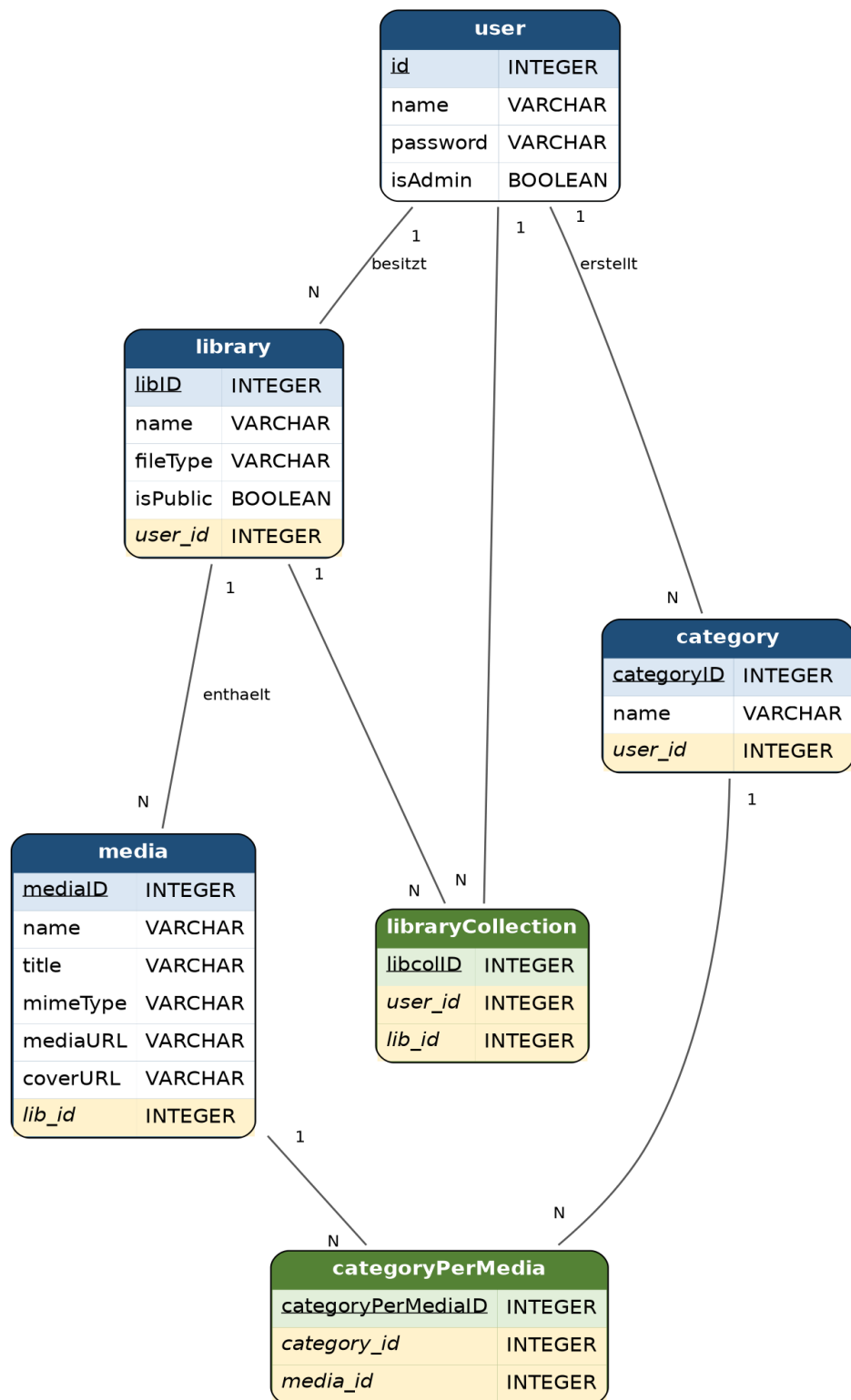
1. Projektbeschreibung – Datenbankteil

Bibliothicc ist eine Medienbibliotheks-Anwendung mit einem WPF/C#-Frontend und einem FastAPI-Backend (Python). Die Datenpersistenz erfolgt über eine SQLite-Datenbank, die mit SQLAlchemy als ORM angebunden ist (siehe `database.py` / `models.py`). Ziel der Datenbank ist es, Benutzer, ihre Medienbibliotheken (Sammlungen von Video-, Bild-, Text- oder Audiodateien), die enthaltenen Mediendateien sowie benutzerdefinierte Kategorien zur Klassifizierung der Medien zu verwalten.

Die Datenbank besteht aus sechs Tabellen: `user`, `library`, `libraryCollection`, `media`, `category` und `categoryPerMedia`. Dabei bilden `libraryCollection` und `categoryPerMedia` reine Zuordnungstabellen, über die n:m-Beziehungen abgebildet werden (ein Benutzer kann mehrere öffentliche Bibliotheken anderer Benutzer abonnieren; ein Medium kann mehreren Kategorien zugeordnet sein).

2. Entity-Relationship-Modell (ERM)

Das folgende ERM zeigt alle sechs Tabellen mit ihren Attributen sowie den Beziehungen zwischen ihnen. Primärschlüssel (PK) sind unterstrichen dargestellt, Fremdschlüssel (FK) kursiv. Die Zuordnungstabellen (grün) lösen die beiden n:m-Beziehungen auf.



Beziehungsübersicht

- user (1) — besitzt — (N) library [user_id in library, nullable: Bibliothek kann auch ohne festen Owner existieren]
- user (N) — abonniert — (M) library, aufgelöst über libraryCollection
- library (1) — enthält — (N) media
- user (1) — erstellt — (N) category
- category (N) — klassifiziert — (M) media, aufgelöst über categoryPerMedia

3. Relationenmodell (RM-Übersicht)

3.1 user

Spalte	Datentyp	Schlüssel	Nullable
<u>id</u>	INTEGER	PK	Nein
name	VARCHAR	-	Nein
password	VARCHAR	-	Nein
isAdmin	BOOLEAN	-	Nein

user(id, name, password, isAdmin)

3.2 library

Spalte	Datentyp	Schlüssel	Nullable
<u>libID</u>	INTEGER	PK	Nein
name	VARCHAR	-	Nein
fileType	VARCHAR	-	Nein
isPublic	BOOLEAN	-	Nein
<i>user_id</i>	INTEGER	FK	Ja

library(libID, name, fileType, isPublic, user_id) — *user_id* verweist auf *user(id)*

3.3 libraryCollection

Spalte	Datentyp	Schlüssel	Nullable
<u>libcolID</u>	INTEGER	PK	Nein
<i>user_id</i>	INTEGER	FK	Nein
<i>lib_id</i>	INTEGER	FK	Nein

libraryCollection(libcolID, user_id, lib_id) — Zuordnungstabelle *user* ↔ *library*; *user_id* → *user(id)*, *lib_id* → *library(libID)*

3.4 media

Spalte	Datentyp	Schlüssel	Nullable
<u>mediaID</u>	INTEGER	PK	Nein
name	VARCHAR	-	Nein
title	VARCHAR	-	Nein
mimeType	VARCHAR	-	Nein
mediaURL	VARCHAR	-	Nein
coverURL	VARCHAR	-	Ja
<i>lib_id</i>	INTEGER	FK	Nein

media(mediaID, name, title, mimeType, mediaURL, coverURL, lib_id) — *lib_id* → *library(libID)*

3.5 category

Spalte	Datentyp	Schlüssel	Nullable
<u>categoryID</u>	INTEGER	PK	Nein
name	VARCHAR	-	Nein

Spalte	Datentyp	Schlüssel	Nullable
<i>user_id</i>	INTEGER	FK	Nein

category(categoryID, name, user_id) — user_id → user(id)

3.6 categoryPerMedia

Spalte	Datentyp	Schlüssel	Nullable
<u>categoryPerMediaID</u>	INTEGER	PK	Nein
<i>category_id</i>	INTEGER	FK	Nein
<i>media_id</i>	INTEGER	FK	Nein

categoryPerMedia(categoryPerMediaID, category_id, media_id) — Zuordnungstabelle category ↔ media

4. Normalisierungsnachweis (1NF – 3NF)

Alle sechs Tabellen besitzen einen einspaltigen, künstlichen Primärschlüssel (Surrogatschlüssel). Daher kann bei keiner Tabelle eine partielle funktionale Abhängigkeit auftreten — die 2NF ist somit automatisch erfüllt, sobald die 1NF erfüllt ist. Im Folgenden wird dies für jede Tabelle einzeln nachgewiesen.

4.1 user

- 1NF: Alle Attribute (name, password, isAdmin) sind atomar, es gibt keine Wiederholungsgruppen oder mehrwertigen Felder. → erfüllt.
- 2NF: Der Primärschlüssel id besteht nur aus einer Spalte, eine partielle Abhängigkeit ist daher ausgeschlossen. → erfüllt.
- 3NF: name, password und isAdmin beschreiben jeweils direkt den Benutzer und hängen nur von id ab; keines der Attribute hängt von einem anderen Nichtschlüsselattribut ab (keine transitive Abhängigkeit). → erfüllt.

4.2 library

- 1NF: name, fileType, isPublic und user_id sind atomare Werte. → erfüllt.
- 2NF: Einspaltiger Primärschlüssel libID, keine partielle Abhängigkeit möglich. → erfüllt.
- 3NF: Alle Attribute beschreiben direkt die Bibliothek. user_id verweist lediglich auf den Eigentümer, es werden keine abgeleiteten Owner-Daten (z. B. der Benutzername) redundant in dieser Tabelle gespeichert. → erfüllt.

4.3 libraryCollection

- 1NF: user_id und lib_id sind atomare Fremdschlüsselwerte. → erfüllt.
- 2NF: Einspaltiger Primärschlüssel libcolID. → erfüllt.
- 3NF: Die Tabelle enthält außer den beiden Fremdschlüsseln keine weiteren Attribute, transitive Abhängigkeiten sind somit ausgeschlossen. → erfüllt.

4.4 media

- 1NF: Alle Attribute (name, title, mimeType, mediaURL, coverURL, lib_id) sind atomar; es wird z. B. nicht mehr als eine URL pro Feld gespeichert. → erfüllt.
- 2NF: Einspaltiger Primärschlüssel mediaID, keine partielle Abhängigkeit möglich. → erfüllt.
- 3NF: Sämtliche Attribute beschreiben ausschließlich das jeweilige Medienobjekt. lib_id verweist nur auf die zugehörige Bibliothek; Bibliotheksdaten selbst (z. B. der Bibliotheksname) werden nicht redundant in media gespeichert. → erfüllt.

4.5 category

- 1NF: name und user_id sind atomare Werte. → erfüllt.
- 2NF: Einspaltiger Primärschlüssel categoryID. → erfüllt.
- 3NF: name hängt nur von categoryID ab, user_id verweist nur auf den Ersteller, ohne weitere Benutzerdaten zu duplizieren. → erfüllt.

4.6 categoryPerMedia

- 1NF: category_id und media_id sind atomare Fremdschlüsselwerte. → erfüllt.
- 2NF: Einspaltiger Primärschlüssel categoryPerMediaID. → erfüllt.
- 3NF: Keine weiteren Nichtschlüsselattribute vorhanden, daher keine transitive Abhängigkeit möglich. → erfüllt.

4.7 Fazit

Alle sechs Tabellen der Bibliothicc-Datenbank erfüllen die erste, zweite und dritte Normalform. Redundanzen werden konsequent vermieden, n:m-Beziehungen sind über eigenständige Zuordnungstabellen (libraryCollection, categoryPerMedia) korrekt aufgelöst, und jedes Nichtschlüsselattribut hängt ausschließlich vom jeweiligen Primärschlüssel der eigenen Tabelle ab.